1. Complete Campus Smart-Parking Application

CampusSmartParkingSystem.java

```
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Duration;
import java.util.*;


/**
 * Campus Smart-Parking System
 * Console-based Java implementation for Assignment 1 (CO1).
 * Compile: javac CampusSmartParkingSystem.java
 * Run:     java CampusSmartParkingSystem
 */
public class CampusSmartParkingSystem {
    private final Scanner scanner = new Scanner(System.in);
    private final ParkingFacility facility;
    private final Map<String, ParkingTicket> activeTickets = new HashMap<>();
    private final Map<String, ParkingUser> users = new HashMap<>();

    public CampusSmartParkingSystem(int[] slotsPerFloor) {
        facility = new ParkingFacility(slotsPerFloor);
    }

    public static void main(String[] args) {
        CampusSmartParkingSystem app =
            new CampusSmartParkingSystem(new int[]{5, 4, 6});
        app.run();
    }

    private void run() {
        boolean running = true;
        while (running) {
            System.out.println("\n===== CAMPUS SMART-PARKING SYSTEM =====");
            System.out.println("1. Register / update user pass");
            System.out.println("2. Vehicle entry");
            System.out.println("3. Vehicle exit");
            System.out.println("4. View parking availability");
            System.out.println("5. View active tickets");
            System.out.println("0. Exit");
            int choice = readInt("Choose an option: ");
            switch (choice) {
                case 1 -> registerUser();
                case 2 -> vehicleEntry();
                case 3 -> vehicleExit();
                case 4 -> facility.printAvailability();
```

```
            case 5 -> printActiveTickets();
            case 0 -> running = false;
            default -> System.out.println("Invalid option.");
        }
    }
    System.out.println("Thank you for using Campus Smart-Parking System.");
}

private void registerUser() {
    System.out.print("Enter user ID: ");
    String id = scanner.nextLine().trim();
    if (id.isEmpty()) {
        System.out.println("User ID cannot be empty.");
        return;
    }
    System.out.println("User type: 1. Student  2. Staff  3. Visitor");
    int type = readInt("Choose user type: ");
    PassType pass = PassType.NONE;
    if (type == 1 || type == 2) {
        System.out.println("Pass: 1. No pass  2. Monthly pass");
        int p = readInt("Choose pass: ");
        if (p == 2) pass = PassType.MONTHLY;
    } else if (type == 3) {
        System.out.println("Visitor pass: 1. No pass  2. One-day pass");
        int p = readInt("Choose pass: ");
        if (p == 2) pass = PassType.ONE_DAY;
    } else {
        System.out.println("Invalid user type.");
        return;
    }
    users.put(id, new ParkingUser(id, type, pass));
    System.out.println("User details saved.");
}

private void vehicleEntry() {
    System.out.print("Enter user ID: ");
    String userId = scanner.nextLine().trim();
    ParkingUser user = users.get(userId);
    if (user == null) {
        System.out.println("User not found. Register the user first.");
        return;
    }
    System.out.print("Enter vehicle registration number: ");
    String regNo = scanner.nextLine().trim().toUpperCase();
    if (regNo.isEmpty()) {
        System.out.println("Registration number cannot be empty.");
        return;
    }
    if (activeTickets.containsKey(regNo)) {
        System.out.println("This vehicle already has an active ticket.");
```

```
            return;
        }

        System.out.println("Vehicle type: 1. Two-wheeler  2. Car  3. Electric car");
        int type = readInt("Choose vehicle type: ");
        Vehicle vehicle;
        switch (type) {
            case 1 -> vehicle = new TwoWheeler(regNo);
            case 2 -> vehicle = new Car(regNo);
            case 3 -> vehicle = new ElectricCar(regNo);
            default -> {
                System.out.println("Unknown vehicle type.");
                return;
            }
        }

        int[] position = facility.allocate(regNo);
        if (position == null) {
            System.out.println("Parking facility is full.");
            return;
        }
        long ticketNo = TicketNumberGenerator.next();
        ParkingTicket ticket = new ParkingTicket(ticketNo, vehicle, user,
            position[0], position[1], LocalDateTime.now());
        activeTickets.put(regNo, ticket);

        System.out.println("Vehicle entry successful.");
        System.out.println("Ticket number: " + ticketNo);
        System.out.println("Floor: " + position[0] + ", Slot: " + position[1]);
        System.out.println("Entry time: " + ticket.getEntryTime());
    }

    private void vehicleExit() {
        System.out.print("Enter vehicle registration number: ");
        String regNo = scanner.nextLine().trim().toUpperCase();
        ParkingTicket ticket = activeTickets.get(regNo);
        if (ticket == null) {
            System.out.println("No active ticket found for this vehicle.");
            return;
        }

        long minutes = Duration.between(ticket.getEntryTime(),
            LocalDateTime.now()).toMinutes();
        int duration = (int)Math.max(0, minutes);
        double fee = ticket.calculateParkingFee(duration);

        if (ticket.getVehicle() instanceof Chargeable) {
            System.out.print("Enter charging energy used (kWh), or 0: ");
```

```
            int kWh = readInt("");
            if (kWh > 0) {
                fee += ((Chargeable)ticket.getVehicle()).chargingFee(kWh);
            }
        }

        facility.release(ticket.getFloor(), ticket.getSlot());
        ticket.close(LocalDateTime.now(), fee);
        activeTickets.remove(regNo);

        System.out.println("Vehicle exit successful.");
        System.out.println("Parking duration: " + duration + " minute(s)");
        System.out.printf(Locale.US, "Total fee: Rs %.2f%n", fee);
        System.out.println("Slot released.");
    }

    private void printActiveTickets() {
        if (activeTickets.isEmpty()) {
            System.out.println("No active parking tickets.");
            return;
        }
        for (ParkingTicket ticket : activeTickets.values()) {
            System.out.println(ticket);
        }
    }

    private int readInt(String prompt) {
        while (true) {
            System.out.print(prompt);
            String value = scanner.nextLine().trim();
            try {
                return Integer.parseInt(value);
            } catch (NumberFormatException ex) {
                System.out.println("Please enter a valid whole number.");
            }
        }
    }
}

enum PassType { NONE, MONTHLY, ONE_DAY }

class ParkingUser {
    private final String userId;
    private final int userType; // 1 student, 2 staff, 3 visitor
    private final PassType passType;

    ParkingUser(String userId, int userType, PassType passType) {
        this.userId = Objects.requireNonNull(userId);
```

```
        this.userType = userType;
        this.passType = Objects.requireNonNull(passType);
    }

    boolean hasFreePass() {
        return (userType == 1 || userType == 2) && passType == PassType.MONTHLY;
    }

    boolean hasOneDayPass() {
        return userType == 3 && passType == PassType.ONE_DAY;
    }

    public String getUserId() { return userId; }
    public PassType getPassType() { return passType; }
}

class ParkingFacility {
    private final String[][] slots;

    ParkingFacility(int[] slotsPerFloor) {
        if (slotsPerFloor == null) throw new IllegalArgumentException("Floor data required");
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            if (slotsPerFloor[f] < 0) throw new IllegalArgumentException("Negative slot count");
            slots[f] = new String[slotsPerFloor[f]];
        }
    }

    public int[] allocate(String vehicleNo) {
        if (vehicleNo == null || vehicleNo.isBlank())
            throw new IllegalArgumentException("Vehicle number required");
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
            }
        }
        return null;
    }

    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
```

```
    }

    public void printAvailability() {
        System.out.println("\n--- PARKING AVAILABILITY ---");
        for (int f = 0; f < slots.length; f++) {
            int free = 0;
            for (String value : slots[f]) if (value == null) free++;
            System.out.println("Floor " + f + ": " + free + "/" + slots[f].length + " slot(s) free");
        }
    }
}

abstract class Vehicle {
    private final String regNo;

    protected Vehicle(String regNo) {
        if (regNo == null || regNo.isBlank())
            throw new IllegalArgumentException("Registration number required");
        this.regNo = regNo.trim().toUpperCase();
    }

    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();

    public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Minutes cannot be negative");
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }

    @Override public boolean equals(Object obj) {
        return obj instanceof Vehicle other && regNo.equals(other.regNo);
    }

    @Override public int hashCode() { return regNo.hashCode(); }

    @Override public String toString() {
        return getClass().getSimpleName() + " " + regNo;
    }
}

class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 10.0; }
}

class Car extends Vehicle {
```

```
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30.0; }
}

interface Chargeable {
    double chargingFee(int kWh);
}

class ElectricCar extends Vehicle implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 40.0; }
    @Override public double chargingFee(int kWh) {
        if (kWh < 0) throw new IllegalArgumentException("kWh cannot be negative");
        return kWh * 3.0;
    }
}

class ParkingTicket {
    private static long nextTicketNo = 1001;
    private final long ticketNo;
    private final Vehicle vehicle;
    private final ParkingUser user;
    private final int floor;
    private final int slot;
    private final LocalDateTime entryTime;
    private LocalDateTime exitTime;
    private double finalFee;
    private boolean closed;

    ParkingTicket(long ticketNo, Vehicle vehicle, ParkingUser user,
                  int floor, int slot, LocalDateTime entryTime) {
        if (ticketNo <= 0 || vehicle == null || user == null ||
            floor < 0 || slot < 0 || entryTime == null)
            throw new IllegalArgumentException("Invalid ticket details");
        this.ticketNo = ticketNo;
        this.vehicle = vehicle;
        this.user = user;
        this.floor = floor;
        this.slot = slot;
        this.entryTime = entryTime;
    }

    static synchronized long createTicketNumber() { return nextTicketNo++; }
    public Vehicle getVehicle() { return vehicle; }
    public int getFloor() { return floor; }
    public int getSlot() { return slot; }
    public LocalDateTime getEntryTime() { return entryTime; }
```

```
    double calculateParkingFee(int minutes) {
        if (user.hasFreePass() || user.hasOneDayPass()) return 0.0;
        return vehicle.calculateFee(minutes);
    }

    void close(LocalDateTime exitTime, double fee) {
        this.exitTime = Objects.requireNonNull(exitTime);
        this.finalFee = fee;
        this.closed = true;
    }

    @Override public String toString() {
        return "Ticket " + ticketNo + " | " + vehicle + " | floor " +
            floor + " | slot " + slot + " | entry " + entryTime;
    }
}

class TicketNumberGenerator {
    private static long nextTicket = 1001;
    static synchronized long next() { return nextTicket++; }
}
```

2. Parking Facility – Task 3 Complete the Code

ParkingFacility.java

```
import java.util.Arrays;

public class ParkingFacility {
    private final String[][] slots;

    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            if (slotsPerFloor[f] < 0) throw new IllegalArgumentException("Invalid slot count");
            slots[f] = new String[slotsPerFloor[f]];
        }
    }

    public int[] allocate(String vehicleNo) {
        if (vehicleNo == null || vehicleNo.isBlank())
            throw new IllegalArgumentException("Vehicle number required");
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
```

```
            }
        }
        return null;
    }

    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }

    public static void main(String[] args) {
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(Arrays.toString(p.allocate("TN01AB1234")));
        System.out.println(Arrays.toString(p.allocate("TN02CD5678")));
        System.out.println(Arrays.toString(p.allocate("TN03EF9999")));
        System.out.println(p.allocate("TN04GH4444"));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1)
                + " " + p.release(5, 0));
        System.out.println(Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

3. Vehicle Hierarchy and Fee Demo – Tasks 6 to 9

FeeDemo.java

```
interface Chargeable {
    double chargingFee(int kWh);
}

abstract class Vehicle {
    private final String regNo;

    protected Vehicle(String regNo) { this.regNo = regNo; }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();

    public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Negative minutes");
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }

    @Override public boolean equals(Object o) {
        return o instanceof Vehicle && regNo.equals(((Vehicle)o).regNo);
```

```
    }
    @Override public int hashCode() { return regNo.hashCode(); }
}

class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 10.0; }
}
class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30.0; }
}
class ElectricCar extends Vehicle implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 40.0; }
    @Override public double chargingFee(int kWh) { return kWh * 3.0; }
}

public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] vehicles = {
            new TwoWheeler("TN01A1"),
            new Car("TN02B2"),
            new ElectricCar("TN03C3")
        };
        for (Vehicle v : vehicles) {
            double fee = v.calculateFee(130);
            if (v instanceof Chargeable c) fee += c.chargingFee(10);
            System.out.println(v.getClass().getSimpleName() + " "
                    + v.getRegNo() + " -> Rs " + fee);
        }
        System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));
    }
}
```

4. README – Compile and Run Instructions

README.txt

CAMPUS SMART-PARKING SYSTEM – JAVA SOURCE CODE

Files:

1. CampusSmartParkingSystem.java
   Complete menu-driven console application. It supports user registration, monthly / one-day pass selection, vehicle entry and exit, slot allocation, ticket generation, availability display and fee calculation.

2. ParkingFacility.java
   Standalone completed Task 3 program. It demonstrates jagged-array slot allocation and release. Its expected output matches the assignment PPT.

3. FeeDemo.java
   Standalone completed Vehicle / Chargeable demonstration for Tasks 6–9. Its expected output matches the assignment PPT.

REQUIREMENT:
JDK 17 or later is recommended because the source uses modern Java syntax (switch expressions and pattern matching for instanceof).

RUN THE COMPLETE APPLICATION:
javac CampusSmartParkingSystem.java
java CampusSmartParkingSystem

RUN TASK 3:
javac ParkingFacility.java
java ParkingFacility

RUN TASKS 6–9:
javac FeeDemo.java
java FeeDemo

NOTE:
The complete application is an academic console prototype. It stores data in memory only. It does not use a database, web API, real payment gateway, or persistent storage. The sample hourly rates and charging fee are demonstration values and should be confirmed with your faculty.